

EasyDocument Enterprise Implementation Package

A production-ready software specification package for development teams and AI coding agents.

Field	Value
Product Name	EasyDocument
Country Scope	Nepal only
Date	2026-06-19
Platform	Flutter mobile application for customers and agents; React admin portal
Backend	Node.js with NestJS, PostgreSQL, Redis, Socket.IO, WebRTC signaling
Storage	Self-hosted MinIO
Maps	Google Maps
Payment	Direct payment between customer and agent; no escrow in v1
Language	English only for v1
Scale Target	10,000 active users

This package is intended to be handed to a product manager, UI/UX designer, backend engineer, Flutter engineer, QA engineer, DevOps engineer, and AI coding agent. It defines the build scope, operating rules, data model, APIs, architecture, acceptance criteria, development backlog, and release plan.

Table of Contents

1. 1. Executive Summary and Product Strategy
2. 2. Scope, Assumptions, and Constraints
3. 3. Stakeholders, Personas, and Operating Model
4. 4. Product Requirements
5. 5. Customer Mobile App Specification
6. 6. Agent Mobile App Specification
7. 7. React Admin Portal Specification
8. 8. Task Lifecycle and Business Rules
9. 9. Communication, Chat, Media, and Calls
10. 10. Agent Matching and Location Design
11. 11. Reviews, Ratings, and Reputation
12. 12. Direct Payment Model
13. 13. Dispute Mediation Workflow
14. 14. Notification Matrix
15. 15. Non-Functional Requirements
16. 16. Security, Privacy, and Abuse Prevention
17. 17. Technical Architecture
18. 18. Backend NestJS Module Design
19. 19. Flutter Application Architecture
20. 20. React Admin Architecture
21. 21. PostgreSQL Database Specification
22. 22. API Contract Specification
23. 23. Socket.IO and WebRTC Specification
24. 24. MinIO Object Storage Design
25. 25. DevOps, Environments, and Deployment
26. 26. Testing and Quality Strategy
27. 27. Analytics, Monitoring, and Operations
28. 28. Implementation Backlog and Sprint Plan
29. 29. AI Coding Agent Instructions
30. 30. Appendices: SQL, OpenAPI, Diagrams, Checklists

1. Executive Summary and Product Strategy

EasyDocument is a Nepal-focused mobile platform that connects customers who need document retrieval, document submission, document verification, or office follow-up services with verified local agents. The platform exists to reduce the time, travel burden, and uncertainty involved in dealing with government offices, universities, banks, embassies, municipalities, courts, passport offices, tax offices, and other organizations.

The v1 product is deliberately practical: customers create a request, manually enter the document and organization information, nearby verified agents receive the task, one agent accepts, a dedicated communication room is created, the customer and agent discuss scope and direct payment, the customer confirms the deal, and the agent starts the work. The agent then updates task status until the customer marks the task complete and reviews the agent.

Strategic Area	Decision
Market	Nepal only for v1; optimize for Kathmandu Valley first, then expand by province.
Trust Model	Agents must complete OTP, citizenship upload, selfie upload, permanent location capture, and admin verification.
Payment Model	Direct payment between customer and agent. The platform records agreed amount and payment method but does not hold funds in v1.
Communication	In-app chat, file sharing, audio/video attachments, in-app call, and external call log.
Admin Role	Admin verifies agents, mediates disputes, reviews abuse reports, and monitors task health.
Scale	Designed for 10,000 active users with horizontal API scaling and managed operational discipline.

1.1 Product Vision

To become the trusted digital network for document-related field services across Nepal by combining verified agents, transparent communication, task status tracking, and simple customer experiences.

1.2 Mission

- Help customers avoid unnecessary travel, queues, and repeated office visits.
- Create earning opportunities for verified local agents who understand document processes.
- Provide transparent task tracking and accountability for document retrieval services.
- Build a secure and auditable platform for sensitive document communication.

1.3 Business Goals

Goal	Measurement	Target for v1
Acquire trusted agents	Verified agent count	500 verified agents across key cities
Drive request completion	Completion rate	80 percent of accepted tasks completed
Customer satisfaction	Average customer rating	4.2+ out of 5
Agent reliability	Cancellation rate	Less than 10 percent after acceptance
Operational response	Dispute first response time	Within 24 business hours
System reliability	API uptime	99.5 percent monthly target for v1

2. Scope, Assumptions, and Constraints

2.1 Confirmed Scope

Area	Decision
Country	Nepal only
Customer Platform	Flutter mobile app
Agent Platform	Flutter mobile app
Admin Platform	React web portal
Backend	Node.js NestJS
Database	PostgreSQL with PostGIS extension
Cache and Realtime Support	Redis
Object Storage	Self-hosted MinIO
Maps	Google Maps
Payment	Direct customer-to-agent payment
Call System	In-app call plus external call logging
Organization Data	Manual entry by customer
Agent Availability	Agents are assumed always online unless manually unavailable or suspended
Language	English only
Target Scale	10,000 active users

2.2 Out of Scope for v1

- Escrow, wallet custody, and platform-held funds are out of scope for v1.
- Automated government office integration is out of scope unless a specific partner API exists later.
- Multi-language UI is out of scope for v1; English is the only supported UI language.
- Web customer portal is out of scope; customers and agents use mobile apps only.
- Automated legal advice is out of scope; the app only facilitates logistics and communication.
- Government identity verification API integration is out of scope for v1, but admin verification is required.

2.3 Key Product Assumptions

- Customers are comfortable providing phone number, name, and address during registration.
- Agents are willing to upload citizenship images, a selfie, and location details to become verified.
- Customers and agents can negotiate price and delivery time within chat before starting the task.
- Many document requests will require physical office visits, therefore location proximity matters.
- For early adoption, transparency and trust are more important than heavy automation.

3. Stakeholders, Personas, and Operating Model

Stakeholder	Responsibilities	Primary Interfaces
Customer	Create requests, communicate, confirm deal, pay agent directly, close task, review agent	Flutter mobile app
Agent	Register, complete KYC, accept requests, communicate, update task status, deliver document	Flutter mobile app
Admin	Verify agents, mediate disputes, monitor reports, manage users, inspect audit trail	React admin portal
Operations	Handle daily marketplace health, problematic tasks, and customer support	Admin portal and dashboards
Finance/Ops	Review direct payment issues and commission model when introduced	Admin portal and exports
Engineering	Build, deploy, monitor, and maintain platform services	Repository, CI/CD, monitoring tools

3.1 Customer Personas

Busy Professional

Lives in Kathmandu, does not have time for repeated office visits. Needs passport, tax, or bank documents handled quickly. Values trust and status visibility.

Student Outside Hometown

Studies or works away from home. Needs transcripts, certificates, recommendation letters, or municipality documents collected from a distant office.

Nepali Abroad Family Member

Family member in Nepal uses the app to coordinate local agents for document retrieval on behalf of someone living abroad.

Small Business Owner

Needs tax, registration, municipal, or banking paperwork moved between offices. Values reliability and proof of progress.

3.2 Agent Personas

Experienced Office Runner

Knows government office procedures and earns through repeated document errands.

Local Student Agent

Available near universities, banks, and district offices. Wants flexible income.

Specialized Document Agent

Focuses on passport, PAN, license, transcript, or embassy-related work. Can command higher trust through ratings.

3.3 Operating Model

31. Customer registers with phone OTP, name, and address.
32. Agent registers with phone OTP, citizenship, selfie, permanent address, and permanent location.
33. Admin verifies or rejects agent registration.
34. Customer creates task with document name, organization name, organization location/address, description, and optional attachments.
35. System broadcasts to eligible nearby verified agents based on organization location and service radius.
36. One agent accepts; a task-specific communication room is created.
37. Customer and agent discuss task details, expected delivery date, and direct payment method.
38. Customer confirms the contract; task status becomes ACTIVE.
39. Agent updates status until delivered. Customer verifies and marks completed.

40. Customer rates and reviews the agent. Disputes are handled by admin mediation.

4. Product Requirements

REQ-AUTH-001 - OTP Registration and Login

Field	Value
Priority	P0
Description	The system shall authenticate customers and agents using phone-number OTP.
Acceptance Criteria	User can request OTP, verify OTP, receive JWT tokens, and access role-appropriate screens.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-AUTH-002 - Refresh Token Rotation

Field	Value
Priority	P0
Description	The backend shall issue short-lived access tokens and rotating refresh tokens.
Acceptance Criteria	Expired access token can be refreshed; reused refresh token is rejected and logged.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-CUST-001 - Customer Profile

Field	Value
Priority	P0
Description	Customer shall register with phone number, full name, and address.
Acceptance Criteria	Customer cannot create a task until phone is verified and name is present.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-AGENT-001 - Agent KYC Registration

Field	Value
Priority	P0
Description	Agent shall upload citizenship front, citizenship back, selfie, permanent address, and permanent location.
Acceptance Criteria	Admin receives pending verification item with all required KYC fields.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-AGENT-002 - Agent Verification

Field	Value
Priority	P0
Description	Admin shall approve, reject, or suspend agents.
Acceptance Criteria	Only verified and available agents receive task broadcasts.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-001 - Create Task

Field	Value
Priority	P0
Description	Customer shall create a task by entering document name, organization name, organization address/location, description, and optional attachments.
Acceptance Criteria	Task title uses USER-DOCUMENT-ORGANIZATION format and status becomes CREATED then BROADCASTED.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-002 - Agent Broadcast

Field	Value
Priority	P0
Description	System shall broadcast the request to nearby verified agents around the organization location.
Acceptance Criteria	Eligible agents receive push and in-app notification; broadcasts are stored.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-003 - Agent Accept

Field	Value
Priority	P0
Description	Agent shall accept an available task.
Acceptance Criteria	First successful acceptance assigns the agent and creates a chat room.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-004 - Contract Confirmation

Field	Value
Priority	P0
Description	Customer shall confirm the deal after chat discussion.
Acceptance Criteria	Task moves to ACTIVE only after customer confirmation.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-005 - Expected Delivery Date

Field	Value
Priority	P0
Description	Agent shall set expected delivery date before or during task activation.
Acceptance Criteria	Customer sees expected date on task detail screen and receives notification when updated.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-TASK-006 - Status Updates

Field	Value
Priority	P0
Description	Agent shall update task status with notes.
Acceptance Criteria	Each update is visible to customer and stored in task status history.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-CHAT-001 - Dedicated Chat Room

Field	Value
Priority	P0
Description	Each accepted task shall have one secure room between customer and assigned agent.
Acceptance Criteria	Only task customer, assigned agent, and authorized admin can access messages.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-CHAT-002 - Media Sharing

Field	Value
Priority	P0
Description	Users shall send images, PDFs, audio, and video attachments in

	chat.
Acceptance Criteria	Files are uploaded to MinIO, scanned/validated, and referenced by secure URLs.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-CALL-001 - In-App Calls

Field	Value
Priority	P1
Description	Customer and agent shall be able to initiate in-app audio/video calls.
Acceptance Criteria	WebRTC signaling connects both parties and call logs are stored.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-CALL-002 - External Call Log

Field	Value
Priority	P1
Description	Users shall be able to initiate external phone calls from the task room.
Acceptance Criteria	A call log entry is stored as CALL_LOG message type.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-REVIEW-001 - Rating and Review

Field	Value
Priority	P0
Description	Customer shall rate and review an agent after task completion.
Acceptance Criteria	Review is linked to task and updates agent average rating.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-DISP-001 - Open Dispute

Field	Value
Priority	P0
Description	Customer or agent shall open a dispute for problematic tasks.
Acceptance Criteria	Admin sees dispute in queue and can mediate with notes.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-ADMIN-001 - Admin Dashboard

Field	Value
Priority	P0
Description	Admin shall view agents, tasks, disputes, users, and audit logs.
Acceptance Criteria	Admin can filter queues and take permitted actions based on RBAC.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-NOTIF-001 - Push Notifications

Field	Value
Priority	P0
Description	System shall send push notifications for major events.
Acceptance Criteria	Task created, accepted, message, status change, dispute update all produce notifications.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

REQ-AUDIT-001 - Audit Logs

Field	Value
-------	-------

Priority	P0
Description	Sensitive actions shall create audit log entries.
Acceptance Criteria	Agent verification, user suspension, dispute resolution, and admin edits are logged.
Engineering Notes	Implement with service-level validation, audit logging, and automated tests.

4.1 Product Success Metrics

Metric	Definition	v1 Target	Owner
Task completion rate	Completed tasks divided by accepted tasks	80 percent	Operations
Median agent acceptance time	Median time from broadcast to accept	Under 10 minutes in active cities	Marketplace
Dispute rate	Disputed tasks divided by completed/active tasks	Under 5 percent	Operations
Average agent rating	Weighted average of agent reviews	4.2+	Product
Agent verification SLA	Time from registration to decision	Under 48 hours	Admin
Crash-free sessions	Mobile sessions without crash	99 percent	Engineering
API p95 latency	Critical endpoints p95 response time	Under 400 ms excluding media upload	Backend

5. Customer Mobile App Specification

The customer mobile app shall be built in Flutter and share design primitives with the agent experience while keeping role-specific flows separate. Customers must be able to register quickly, create a document request, communicate with the assigned agent, track progress, close the deal, and review the agent.

Screen	Purpose
Splash	Checks app version, token presence, maintenance mode, and routes to onboarding or dashboard.
OTP Login	Phone number entry, OTP resend timer, verification error handling, terms acceptance.
Customer Profile Setup	Name and address capture; optional profile photo later.
Customer Dashboard	Active tasks, recent tasks, create request CTA, notifications, support entry.
Create Request Step 1	Document name, task category, organization name, description.
Create Request Step 2	Organization address entry, Google Map picker, optional coordinates.
Create Request Step 3	Attachment upload, instructions, urgency, review and submit.
Broadcast Waiting	Shows request sent to nearby agents with cancel option before acceptance.
Task Detail	Assigned agent info, task status, expected delivery date, timeline, payment method, chat button.
Chat Room	Messages, media upload, voice note, call buttons, external call button, contract confirmation panel.
Review Agent	Star rating, review text, issue reporting, submit review.
Dispute Form	Reason, description, attachments, submit to admin mediation.
History	Completed, cancelled, disputed, and active task records.
Notifications	System notifications with deep links to task/chat/dispute.

5.1 Customer Navigation Flow

Customer App Navigation

Splash -> AuthGate -> OTP Login -> Profile Setup -> Dashboard

Dashboard -> Create Request -> Broadcast Waiting -> Task Detail -> Chat Room

Task Detail -> Close Task -> Review Agent -> History

Task Detail -> Open Dispute -> Dispute Detail

Dashboard -> Notifications -> Target Screen

5.2 Customer Task Creation Rules

- Document name is required and should be normalized for the task title.
- Organization name is required because v1 does not maintain a master office registry.
- Organization address is required; coordinates are recommended when Google Maps geocoding succeeds.
- Customer may attach supporting documents, images, or instructions before broadcasting.
- Customer may cancel before any agent accepts without dispute flow.
- After an agent accepts, cancellation requires a reason and is visible to the agent.

CUST-US-001 - customer

As a customer, I want customer capability 1 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-002 - customer

As a customer, I want customer capability 2 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.

- Acceptance: API failures are recoverable without data loss.

CUST-US-003 - customer

As a customer, I want customer capability 3 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-004 - customer

As a customer, I want customer capability 4 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-005 - customer

As a customer, I want customer capability 5 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-006 - customer

As a customer, I want customer capability 6 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-007 - customer

As a customer, I want customer capability 7 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-008 - customer

As a customer, I want customer capability 8 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-009 - customer

As a customer, I want customer capability 9 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-010 - customer

As a customer, I want customer capability 10 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-011 - customer

As a customer, I want customer capability 11 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-012 - customer

As a customer, I want customer capability 12 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-013 - customer

As a customer, I want customer capability 13 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-014 - customer

As a customer, I want customer capability 14 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

CUST-US-015 - customer

As a customer, I want customer capability 15 in the EasyDocument app, so that I can complete my document-related task with transparency and control.

- Acceptance: The screen validates required fields.
- Acceptance: The action shows loading and clear error states.
- Acceptance: Successful action updates task state and notification state.
- Acceptance: API failures are recoverable without data loss.

6. Agent Mobile App Specification

The agent mobile app is the operational tool for verified workers. It must support identity verification, task broadcast review, task acceptance, communication, status updates, and performance tracking.

Screen	Purpose
Agent OTP Login	Phone authentication shared with customer auth flow.
Agent KYC Step 1	Full name, phone, permanent address, citizenship number.
Agent KYC Step 2	Citizenship front upload, citizenship back upload, selfie photo.
Agent KYC Step 3	Permanent location capture using device location permission and Google Map confirmation.
Verification Pending	Explains admin review status and expected verification timeline.
Agent Dashboard	Incoming requests, active tasks, earnings summary, rating summary.
Request Detail	Task description, organization location, distance, customer request details, accept button.
Agent Task Detail	Status update controls, expected delivery date, chat, call, document delivery actions.
Status Update	Update task status with structured status and notes.
Agent Profile	KYC status, rating, completed tasks, service tags, availability toggle.
Agent Reviews	List of customer reviews and performance guidance.

6.1 Agent KYC Rules

- Agent must provide citizenship number, citizenship front image, citizenship back image, selfie, permanent address, and permanent location.
- Agent permanent location is captured once during registration and can be updated only through a controlled profile update process.
- Agent current location may update when the app is active and permission is granted, but permanent location remains the identity anchor.
- Agent cannot accept tasks until admin status is VERIFIED.
- Agent profile must display verification status clearly.

6.2 Agent Status Update Options

Agent Action	Resulting Status	Visible to Customer	Notification
Accept request	ACCEPTED	Yes	Customer notified
Set expected delivery date	ACCEPTED or ACTIVE	Yes	Customer notified
Confirm work started	IN_PROGRESS	Yes	Customer notified
Document submitted at office	SUBMITTED	Yes	Customer notified
Document collected or ready	DELIVERED	Yes	Customer notified
Request issue support	DISPUTED	Yes	Admin notified

AGENT-US-001 - agent

As a agent, I want agent capability 1 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-002 - agent

As a agent, I want agent capability 2 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-003 - agent

As a agent, I want agent capability 3 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-004 - agent

As a agent, I want agent capability 4 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-005 - agent

As a agent, I want agent capability 5 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-006 - agent

As a agent, I want agent capability 6 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-007 - agent

As a agent, I want agent capability 7 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-008 - agent

As a agent, I want agent capability 8 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-009 - agent

As a agent, I want agent capability 9 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-010 - agent

As a agent, I want agent capability 10 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-011 - agent

As a agent, I want agent capability 11 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-012 - agent

As a agent, I want agent capability 12 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-013 - agent

As a agent, I want agent capability 13 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-014 - agent

As a agent, I want agent capability 14 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.
- Acceptance: All errors are visible and actionable.

AGENT-US-015 - agent

As a agent, I want agent capability 15 in the EasyDocument app, so that I can manage document work professionally and get better ratings.

- Acceptance: Agent-only actions require VERIFIED status.
- Acceptance: Task update is stored with timestamp and actor.
- Acceptance: Customer receives realtime and push notification.

- Acceptance: All errors are visible and actionable.

7. React Admin Portal Specification

The admin portal shall be built in React with TypeScript and a component library such as Material UI. Admin users do not use the mobile app for operations. The portal is the control plane for agent verification, dispute mediation, customer support, task monitoring, user moderation, and audit review.

Module	Requirements
Admin Auth	Admin login, role-based session, MFA optional for later.
Overview Dashboard	Counts of active tasks, pending agents, disputes, messages flagged, and system health.
Agent Verification Queue	Review citizenship images, selfie, location, phone, approve/reject/suspend.
User Management	View customers and agents, status, task history, warnings, suspension.
Task Monitor	Filter by status, date, district, agent, customer, dispute state.
Dispute Center	Review dispute details, chat transcript, attachments, notes, mediation decision.
Audit Logs	Search admin actions and sensitive state changes.
Analytics	Completion rate, acceptance time, active agents, rating trends, disputes.
Settings	System configuration such as broadcast radius, upload limits, notification templates.

7.1 Admin RBAC Matrix

Capability	ADMIN	OPS	SUPPORT	FINANCE
View dashboard	Yes	Yes	Yes	Limited
Approve/reject agents	Yes	Yes	No	No
Suspend users	Yes	No	No	No
View chat transcript for dispute	Yes	Yes	Yes if assigned	No
Resolve dispute	Yes	Yes	No	No
View payment metadata	Yes	Limited	No	Yes
Export analytics	Yes	Yes	No	Yes
View audit logs	Yes	No	No	No
Change system settings	Yes	No	No	No

8. Task Lifecycle and Business Rules

8.1 Task State Machine

CREATED

- > BROADCASTED
- > ACCEPTED
- > NEGOTIATING
- > CONTRACT_PENDING
- > ACTIVE
- > IN_PROGRESS
- > SUBMITTED
- > DELIVERED
- > COMPLETED

Any active non-terminal state may transition to DISPUTED by customer, agent, or admin.

CREATED/BROADCASTED can transition to CANCELLED by customer.

ACCEPTED or later can transition to CANCELLED only with reason and audit trail.

8.2 Task Naming Rule

Task display title must follow the pattern CUSTOMER-DOCUMENT-ORGANIZATION. The system should sanitize spaces and special characters for the code while displaying readable text in the UI.

Input	Example
Customer Name	Rijan Kafle
Document Name	Passport Renewal
Organization Name	Department of Passports
Readable Title	Rijan Kafle - Passport Renewal - Department of Passports
Code Form	RIJANKAFLE-PASSPORTRENEWAL-DEPARTMENTOFPASSPORTS

8.3 State Transition Table

From	To	Actor	Condition
CREATED	BROADCASTED	System	After task is saved and matching is run
BROADCASTED	ACCEPTED	Agent	First eligible agent accepts
ACCEPTED	NEGOTIATING	System	Chat room opens for discussion
NEGOTIATING	CONTRACT_PENDING	Agent or Customer	Agreement details entered
CONTRACT_PENDING	ACTIVE	Customer	Customer confirms deal
ACTIVE	IN_PROGRESS	Agent	Agent starts actual work
IN_PROGRESS	SUBMITTED	Agent	Document submitted or office action completed
SUBMITTED	DELIVERED	Agent	Document retrieved/delivered or result ready
DELIVERED	COMPLETED	Customer	Customer verifies and closes task
Any non-terminal	DISPUTED	Customer/Agent/Admin	Issue requires mediation
CREATED/BROADCASTED	CANCELLED	Customer	No agent has accepted yet

8.4 Edge Cases

- If two agents accept at the same time, backend must use a database transaction or unique task assignment lock so only one acceptance succeeds.
- If the agent loses connectivity after acceptance, customer can open support or cancel with admin review depending on state.
- If the customer closes the task accidentally, admin may reopen only within an operational policy window.
- If documents are sensitive, users should be warned not to share unnecessary information in chat.
- If direct payment fails, the platform records the dispute but is not financially liable unless future policy changes.

9. Communication, Chat, Media, and Calls

Communication is central to the trust model. Every accepted task creates exactly one dedicated room between customer and assigned agent. The room supports text, media, documents, voice notes, video attachments, call logs, in-app calls, and external phone call initiation.

Message Type	Storage	Validation	Retention
TEXT	PostgreSQL messages.body	Max length, profanity/abuse hooks later	Retain with task
IMAGE	MinIO object plus DB metadata	MIME allowlist, size limit	Retain unless deleted by policy
DOCUMENT	MinIO object plus DB metadata	PDF/JPEG/PNG allowlist for v1	Retain with task
AUDIO	MinIO object plus DB metadata	MIME and duration limit	Retain with task
VIDEO	MinIO object plus DB metadata	MIME and size limit	Retain with task
CALL_LOG	PostgreSQL message and call_sessions	Call type and timestamp	Retain with task
SYSTEM	PostgreSQL messages.body	Generated by backend only	Retain with task

9.1 Socket.IO Events

Event	Direction	Payload	Purpose
room.join	Client -> Server	roomId	Join authorized task room
message.send	Client -> Server	roomId, type, body, attachmentId	Send chat message
message.created	Server -> Client	message object	Realtime delivery
message.read	Client -> Server	roomId, messageId	Read receipt
typing.start	Client -> Server	roomId	Typing indicator
typing.stop	Client -> Server	roomId	Typing stopped
task.status.updated	Server -> Client	taskId, status, note	Realtime status update
call.offer	Client -> Server	roomId, sdp	WebRTC offer
call.answer	Client -> Server	roomId, sdp	WebRTC answer
call.ice	Client -> Server	roomId, candidate	ICE candidate exchange

9.2 WebRTC Requirements

- Use backend Socket.IO signaling for offer, answer, ICE candidates, call status, and call log events.
- Start with peer-to-peer WebRTC for v1; evaluate TURN server if NAT traversal failure rate is high.
- Store only call metadata, not call recordings, unless explicit future compliance requirements exist.
- External call option opens the device dialer and creates a call log event in the task chat.

10. Agent Matching and Location Design

The matching engine uses the manually entered organization location when available. If coordinates are missing, the system can broadcast to agents in a broader configured region but should warn the customer that map accuracy is lower.

10.1 Matching Inputs

Input	Source	Required	Use
Organization latitude/longitude	Google Maps picker/geocoder	Preferred	Primary proximity calculation
Organization address text	Customer	Required	Fallback search and display
Agent current location	Agent app when permission granted	Preferred	Nearest agent calculation
Agent permanent location	Agent registration	Required	Fallback and verification anchor
Agent availability	Agent profile	Required	Only available agents receive requests
Agent status	Admin verification	Required	Only VERIFIED agents receive requests
Agent rating	Reviews	Optional	Ranking score
Completion rate	Task history	Optional	Ranking score

10.2 Ranking Formula v1

The initial ranking formula should be simple, explainable, and tunable. It should not over-optimize before marketplace data exists.

```
ranking_score =  
  (distance_score * 0.40) +  
  (rating_score * 0.25) +  
  (completion_score * 0.20) +  
  (response_score * 0.10) +  
  (freshness_score * 0.05)
```

Only agents with status VERIFIED and is_available = true are eligible.

10.3 Broadcast Rules

- Initial radius: 5 km in dense cities; configurable by admin settings.
- If fewer than 5 eligible agents are found, expand to 10 km, then 20 km depending on geography.
- Agent receives task summary, organization location, approximate distance, and accept button.
- Task assignment is first-accept-wins in v1.
- Future version may support proposals or customer-selected agents.

11. Reviews, Ratings, and Reputation

Reviews are a core marketplace trust mechanism. Customers can rate agents after task completion. Agents cannot rate customers in v1, but abuse reports and dispute notes can protect agents from malicious customers.

Rating Dimension	v1 Support	Notes
Overall star rating	Yes	1 to 5 stars required after completion
Written review	Yes	Optional but encouraged
Response speed	Derived	Can be calculated later from message and acceptance times
Completion rate	Derived	Completed tasks divided by accepted tasks
Cancellation rate	Derived	Cancelled tasks after acceptance
Dispute count	Derived	Visible to admin; not necessarily public

11.1 Reputation Update Rule

- After review submission, recalculate agent average rating using all non-deleted reviews.
- Disputed reviews may be hidden by admin if abuse is proven.
- Agent public profile should show average rating, completed task count, and selected recent reviews.
- Review creation must be idempotent per task; one review per completed task.

12. Direct Payment Model

EasyDocument v1 does not hold money. Customers and agents agree on amount and method in chat. The system records agreed amount and selected method for transparency, dispute context, and future analytics. The platform should clearly state that payment is handled directly between customer and agent.

Payment Method	System Role	Risk Handling
Cash	Record method only	Dispute evidence relies on chat and receipts
eSewa	Record method and optional transaction note	No API settlement in v1
Khalti	Record method and optional transaction note	No API settlement in v1
ConnectIPS	Record method and optional transaction note	No API settlement in v1
Bank Transfer	Record method and optional bank receipt attachment	No API settlement in v1

12.1 Payment UX Rules

- Payment amount is not mandatory at request creation; it is agreed after agent accepts.
- Before customer confirms contract, UI should show agreed amount, method, expected delivery date, and important notes.
- Customer must explicitly confirm that payment is direct and outside platform custody.
- Payment disputes can be mediated by admin using chat, attachment, and status history evidence.

13. Dispute Mediation Workflow

Admin mediation provides trust without requiring financial custody. Disputes may involve delayed delivery, incorrect document, payment misunderstanding, agent unresponsiveness, customer cancellation disagreement, or inappropriate behavior.

Dispute Step	Actor	System Action
Open dispute	Customer or Agent	Create dispute, mark task DISPUTED, notify other party and admin
Initial review	Admin/Ops	Read task details, chat transcript, attachments, status history
Request evidence	Admin	Set dispute WAITING_CUSTOMER or WAITING_AGENT
Mediation	Admin	Add internal notes and propose resolution
Decision	Admin	Resolve with notes; optionally warn/suspend user/agent
Close	Admin	Close dispute and restore/complete/cancel task as appropriate

13.1 Dispute Reasons

- Agent not responding
- Customer not responding
- Document not delivered
- Wrong document
- Payment disagreement
- Office rejected request
- Fraud suspicion
- Unsafe behavior
- Other

14. Notification Matrix

Event	Recipient	Channel	Deep Link	Template
OTP sent	Customer/Agent	SMS	Auth	OTP has been sent to your phone.
Agent registration submitted	Agent	Push/In-app	KYC	Your verification is pending admin review.
Agent approved	Agent	Push/In-app	KYC	Your agent profile has been approved.
Task broadcast	Agent	Push/In-app	Task	New document request near you.
Task accepted	Customer	Push/In-app	Task	An agent accepted your request.
Message received	Customer/Agent	Push/In-app	Chat	New message in task chat.
Expected date updated	Customer	Push/In-app	Task	Agent updated expected delivery date.
Status updated	Customer	Push/In-app	Task	Task status has changed.
Task delivered	Customer	Push/In-app	Task	Agent marked document delivered.
Task completed	Agent	Push/In-app	Task	Customer completed the task.
Review received	Agent	Push/In-app	Review	You received a new review.
Dispute opened	Admin/Opposite party	Push/In-app/Admin	Dispute	A dispute has been opened.
Dispute updated	Customer/Agent	Push/In-app	Dispute	Admin updated your dispute.

14.1 Notification Rules

- Push notifications must include data payload for deep linking.
- Notification creation must be transactional with critical state changes when possible.
- SMS is required for OTP and optional for critical alerts in v1.
- In-app notification list must be queryable and mark-as-read capable.

15. Non-Functional Requirements

Category	Requirement	Target
Availability	Core API should remain available under normal operations	99.5 percent monthly v1 target
Performance	Critical API p95 response time excluding file upload	Under 400 ms
Scalability	Support 10,000 active users and bursty broadcast events	Horizontal API replicas and Redis
Security	All sensitive endpoints require JWT and RBAC	100 percent protected endpoints
Privacy	PII and document attachments must be access-controlled	Signed URLs and authorization checks
Reliability	State transitions must be auditable and recoverable	Status history and audit logs
Maintainability	Clear modular architecture with tests	80 percent service layer coverage
Observability	API, database, Redis, MinIO, and Socket.IO metrics	Dashboards and alerts
Backup	PostgreSQL and MinIO backup policy	Daily backup minimum for production
Compatibility	Flutter app supports modern Android and iOS versions	Defined by release target at implementation

16. Security, Privacy, and Abuse Prevention

16.1 Authentication and Authorization

- Use OTP for phone verification and login.
- Use JWT access tokens with short TTL and refresh tokens with rotation.
- Use RBAC on backend; never rely on frontend role checks only.
- Admin endpoints require admin role and audit logging.
- File download endpoints must verify task participant/admin authorization before generating MinIO signed URL.

16.2 Sensitive Data

Data	Risk	Control
Citizenship images	Identity theft	MinIO private bucket, signed URL, admin-only access
Selfie photo	Biometric-like personal image	Private bucket and strict access
Phone number	Privacy and spam	Hash in logs, mask in admin where possible
Chat attachments	Sensitive document content	Access only to task participants/admin during dispute
Location	Safety and privacy	Use minimum needed precision for display and matching

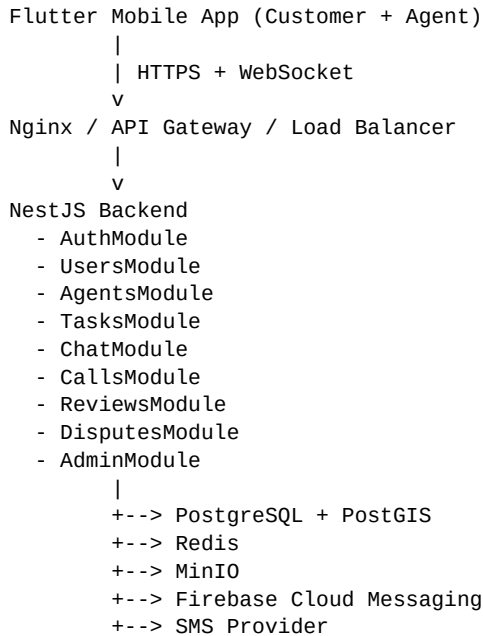
16.3 Abuse Prevention

- Rate-limit OTP requests by phone number, IP address, and device fingerprint.
- Rate-limit task creation to prevent spam broadcasts.
- Allow users to report abusive messages or suspicious behavior.
- Admin can suspend users or agents with audit trail.
- Prevent agents from receiving tasks before verification approval.
- Block file types not explicitly allowed.

17. Technical Architecture

The architecture is modular monolith first with microservice-ready boundaries. For 10,000 active users, a well-structured NestJS backend, PostgreSQL, Redis, MinIO, and horizontal API replicas are sufficient. Premature microservices should be avoided unless operational needs justify them.

High-Level Architecture



React Admin Portal -> Same API Gateway -> Admin APIs

17.1 Architecture Principles

- Modular monolith first, service boundaries clear enough to split later.
- PostgreSQL is the system of record; Redis is cache, queue support, and realtime helper only.
- MinIO stores binary files; PostgreSQL stores metadata and permissions.
- All state transitions go through backend services, never direct database writes from clients.
- Every sensitive action must be auditable.
- Use feature flags for risky rollout areas such as WebRTC and broadcast tuning.

18. Backend NestJS Module Design

Module	Responsibilities
AuthModule	OTP send/verify, JWT issue/refresh, session logout, rate limiting
UsersModule	Customer profile, user status, role metadata
AgentsModule	Agent KYC registration, profile, location, availability, verification status
TasksModule	Task create, broadcast, accept, contract confirm, status updates, completion, cancellation
MatchingModule	Agent eligibility, geospatial ranking, broadcast creation
ChatModule	Chat rooms, messages, attachments, read receipts, Socket.IO events
CallsModule	WebRTC signaling, call session logs, external call logs
ReviewsModule	Ratings, reviews, reputation aggregation
DisputesModule	Open dispute, admin mediation notes, resolution state
NotificationsModule	Push and in-app notification creation and dispatch
FilesModule	MinIO upload policy, signed URL generation, file validation
AdminModule	Admin dashboards, moderation, verification, audit views
AuditModule	Centralized audit log writer
HealthModule	Readiness and liveness checks

18.1 Backend Folder Structure

```
backend/  
  src/  
    main.ts  
    app.module.ts  
    config/  
    common/  
      decorators/  
      guards/  
      interceptors/  
      filters/  
      dto/  
    modules/  
      auth/  
      users/  
      agents/  
      tasks/  
      matching/  
      chat/  
      calls/  
      reviews/  
      disputes/  
      notifications/  
      files/  
      admin/  
      audit/  
      health/  
    database/  
      migrations/  
      seeds/  
    integrations/  
      sms/  
      firebase/  
      minio/  
      google-maps/  
  test/  
  Dockerfile  
  package.json
```

18.2 Backend Coding Standards

- Use DTO classes with validation decorators for all incoming requests.
- Use service classes for business rules and keep controllers thin.
- Use transactions for task acceptance, task status transitions, and review aggregation.
- Use repository or ORM abstraction consistently; do not mix raw SQL randomly except for geospatial matching queries.

- Every module must include unit tests for service methods and e2e tests for critical routes.
- Use structured logging with request ID, user ID, route, and latency.

19. Flutter Application Architecture

The Flutter application will support both customer and agent experiences using role-aware navigation. It should use Clean Architecture, BLoC/Cubit for state management, Dio for HTTP, Socket.IO client for realtime, and GoRouter for navigation.

```
mobile/  
  lib/  
    main.dart  
    app/  
      router/  
      theme/  
      di/  
      config/  
    core/  
      network/  
      storage/  
      errors/  
      widgets/  
      permissions/  
    features/  
      auth/  
        data/ domain/ presentation/  
      customer_home/  
      agent_home/  
      task_create/  
      task_detail/  
      chat/  
      calls/  
      reviews/  
      disputes/  
      notifications/  
      profile/  
    test/  
    integration_test/  
  pubspec.yaml
```

19.1 Flutter Feature Modules

Feature	Screens	State Objects
Auth	OTP login, verify OTP, profile setup	AuthCubit, OtpTimerCubit
Customer Home	Dashboard, history, notifications	CustomerHomeBloc
Agent Home	Dashboard, incoming requests, active tasks	AgentHomeBloc
Task Create	Step forms, map picker, upload	TaskCreateBloc
Task Detail	Timeline, agent info, status, contract	TaskDetailBloc
Chat	Message list, upload, voice note, call buttons	ChatBloc, SocketCubit
Calls	Incoming call, outgoing call, call status	CallCubit
Reviews	Rating form and review list	ReviewBloc
Disputes	Open dispute, dispute detail	DisputeBloc

19.2 Mobile UX Requirements

- Every network action must have loading, success, empty, and error states.
- Create request flow must preserve draft locally if the app closes before submission.
- Chat must support offline-safe retry for unsent messages using client_message_id.
- File upload must show progress and support cancel/retry.
- Location permission must explain why location is needed before asking OS permission.
- Agent KYC form must prevent submission until all required files and location are present.

20. React Admin Architecture

```
admin/  
  src/  
    main.tsx  
    app/  
      router.tsx  
      providers.tsx  
      theme.ts  
    components/  
    features/  
      auth/  
      dashboard/  
      agents/  
      users/  
      tasks/  
      disputes/  
      audit/  
      settings/  
    services/  
      apiClient.ts  
      authService.ts  
    hooks/  
    types/  
  package.json  
  Dockerfile
```

20.1 Admin Technical Requirements

- Use TypeScript strict mode.
- Use role-based route guards.
- Use a shared API client with token refresh.
- Use server-side pagination for large tables.
- Use filters for status, date range, district/address text, agent, customer, and task code.
- Admin actions must show confirmation dialogs for irreversible changes.

21. PostgreSQL Database Specification

PostgreSQL is the system of record. PostGIS is used for geospatial matching. The schema must support task lifecycle auditability, sensitive identity data, chat history, direct payment metadata, dispute mediation, and review aggregation.

Table	Purpose
users	Stores customer, agent, and admin login identity records.
otp_verifications	Stores hashed OTP records and verification attempts.
agent_profiles	Stores KYC and operational data for agents.
agent_service_tags	Stores optional expertise tags for agent search and ranking.
tasks	Stores document requests and assignment state.
task_broadcasts	Stores which agents received which task broadcast.
task_status_history	Stores every status transition for audit and timeline.
chat_rooms	Stores task-specific communication room metadata.
messages	Stores chat message metadata and attachment references.
call_sessions	Stores in-app and external call metadata.
reviews	Stores customer review and rating for completed task.
disputes	Stores admin mediation cases.
notifications	Stores in-app notification records.
audit_logs	Stores sensitive admin and system action audit trail.

21.1 Table Details

users

Stores customer, agent, and admin login identity records.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

otp_verifications

Stores hashed OTP records and verification attempts.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

agent_profiles

Stores KYC and operational data for agents.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

agent_service_tags

Stores optional expertise tags for agent search and ranking.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

tasks

Stores document requests and assignment state.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

task_broadcasts

Stores which agents received which task broadcast.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

task_status_history

Stores every status transition for audit and timeline.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

chat_rooms

Stores task-specific communication room metadata.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

messages

Stores chat message metadata and attachment references.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

call_sessions

Stores in-app and external call metadata.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

reviews

Stores customer review and rating for completed task.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

disputes

Stores admin mediation cases.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

notifications

Stores in-app notification records.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

audit_logs

Stores sensitive admin and system action audit trail.

Design Requirement	Implementation Guidance
Primary key	UUID generated server-side.
Timestamps	created_at and updated_at where records are mutable.
Indexes	Add indexes for foreign keys, status filters, and time-based queries.
Audit	Sensitive changes must be tracked in audit_logs or domain history table.
Deletion	Prefer soft delete or status change for user-facing records; do not hard delete task history.

21.2 SQL Schema Excerpt

```
-- EasyDocument PostgreSQL Schema - Implementation Baseline
-- Target: PostgreSQL 15+, Nepal-only deployment, 10,000 active users
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TYPE user_role AS ENUM ('CUSTOMER', 'AGENT', 'ADMIN', 'OPS', 'FINANCE', 'SUPPORT');
CREATE TYPE user_status AS ENUM ('PENDING_OTP', 'ACTIVE', 'SUSPENDED', 'DELETED');
CREATE TYPE agent_status AS ENUM ('DRAFT', 'PENDING_VERIFICATION', 'VERIFIED', 'REJECTED', 'SUSPENDED');
CREATE TYPE task_status AS ENUM
('CREATED', 'BROADCASTED', 'ACCEPTED', 'NEGOTIATING', 'CONTRACT_PENDING', 'ACTIVE', 'IN_PROGRESS', 'SUBMITTED', 'DE
LIVERED', 'COMPLETED', 'CANCELLED', 'DISPUTED');
CREATE TYPE message_type AS ENUM ('TEXT', 'IMAGE', 'DOCUMENT', 'AUDIO', 'VIDEO', 'SYSTEM', 'CALL_LOG');
CREATE TYPE payment_method AS ENUM ('CASH', 'ESEWA', 'KHALTI', 'CONNECTIPS', 'BANK_TRANSFER', 'OTHER');
CREATE TYPE dispute_status AS ENUM
('OPEN', 'UNDER_REVIEW', 'WAITING_CUSTOMER', 'WAITING_AGENT', 'RESOLVED', 'CLOSED');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  phone_number VARCHAR(20) NOT NULL UNIQUE,
  full_name VARCHAR(120) NOT NULL,
  address_text TEXT,
  role user_role NOT NULL DEFAULT 'CUSTOMER',
  status user_status NOT NULL DEFAULT 'PENDING_OTP',
  profile_photo_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_login_at TIMESTAMPTZ
);

CREATE TABLE otp_verifications (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  phone_number VARCHAR(20) NOT NULL,
  otp_hash TEXT NOT NULL,
  purpose VARCHAR(40) NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  consumed_at TIMESTAMPTZ,
  attempts INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_otp_phone_purpose ON otp_verifications(phone_number, purpose, expires_at DESC);

CREATE TABLE agent_profiles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id),
  citizenship_number VARCHAR(80) NOT NULL,
  citizenship_front_url TEXT NOT NULL,
  citizenship_back_url TEXT NOT NULL,
  selfie_url TEXT NOT NULL,
  permanent_address_text TEXT NOT NULL,
  permanent_location GEOGRAPHY(POINT,4326),
  current_location GEOGRAPHY(POINT,4326),
```

```

        status agent_status NOT NULL DEFAULT 'PENDING_VERIFICATION',
        verification_notes TEXT,
        average_rating NUMERIC(3,2) NOT NULL DEFAULT 0,
        completed_task_count INT NOT NULL DEFAULT 0,
        cancelled_task_count INT NOT NULL DEFAULT 0,
        is_available BOOLEAN NOT NULL DEFAULT TRUE,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );
CREATE INDEX idx_agent_status_available ON agent_profiles(status, is_available);
CREATE INDEX idx_agent_current_location ON agent_profiles USING GIST(current_location);

CREATE TABLE agent_service_tags (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    agent_id UUID NOT NULL REFERENCES agent_profiles(id) ON DELETE CASCADE,
    tag VARCHAR(80) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(agent_id, tag)
);

CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_code VARCHAR(80) NOT NULL UNIQUE,
    customer_id UUID NOT NULL REFERENCES users(id),
    assigned_agent_id UUID REFERENCES agent_profiles(id),
    document_name VARCHAR(120) NOT NULL,
    organization_name VARCHAR(160) NOT NULL,
    organization_address_text TEXT,
    organization_location GEOGRAPHY(POINT,4326),
    task_title VARCHAR(300) NOT NULL,
    description TEXT,
    status task_status NOT NULL DEFAULT 'CREATED',
    direct_payment_method payment_method,
    agreed_amount_paisa BIGINT,
    agent_expected_delivery_at TIMESTAMPTZ,
    customer_closed_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    cancelled_at TIMESTAMPTZ,
    cancellation_reason TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_tasks_customer ON tasks(customer_id, created_at DESC);
CREATE INDEX idx_tasks_agent ON tasks(assigned_agent_id, created_at DESC);
CREATE INDEX idx_tasks_status ON tasks(status, created_at DESC);
CREATE INDEX idx_tasks_org_location ON tasks USING GIST(organization_location);

CREATE TABLE task_broadcasts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    agent_id UUID NOT NULL REFERENCES agent_profiles(id),
    distance_meters INT,
    ranking_score NUMERIC(8,2),
    sent_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    seen_at TIMESTAMPTZ,
    accepted_at TIMESTAMPTZ,
    declined_at TIMESTAMPTZ,
    UNIQUE(task_id, agent_id)
);
CREATE INDEX idx_task_broadcast_agent ON task_broadcasts(agent_id, sent_at DESC);

CREATE TABLE task_status_history (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    old_status task_status,
    new_status task_status NOT NULL,
    changed_by_user_id UUID REFERENCES users(id),

```

```

        note TEXT,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );
CREATE INDEX idx_status_history_task ON task_status_history(task_id, created_at DESC);

CREATE TABLE chat_rooms (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_id UUID NOT NULL UNIQUE REFERENCES tasks(id) ON DELETE CASCADE,
    customer_id UUID NOT NULL REFERENCES users(id),
    agent_user_id UUID NOT NULL REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    archived_at TIMESTAMPTZ
);

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    room_id UUID NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
    sender_id UUID NOT NULL REFERENCES users(id),
    message_type message_type NOT NULL DEFAULT 'TEXT',
    body TEXT,
    attachment_url TEXT,
    attachment_mime VARCHAR(120),
    attachment_size_bytes BIGINT,
    client_message_id VARCHAR(120),
    read_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);
CREATE INDEX idx_messages_room_time ON messages(room_id, created_at DESC);

CREATE TABLE call_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    room_id UUID NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
    initiated_by_user_id UUID NOT NULL REFERENCES users(id),
    call_type VARCHAR(20) NOT NULL CHECK (call_type IN ('AUDIO', 'VIDEO', 'EXTERNAL')),
    status VARCHAR(20) NOT NULL CHECK (status IN ('RINGING', 'ACCEPTED', 'MISSED', 'ENDED', 'FAILED')),
    started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ended_at TIMESTAMPTZ,
    duration_seconds INT
);

CREATE TABLE reviews (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_id UUID NOT NULL UNIQUE REFERENCES tasks(id),
    customer_id UUID NOT NULL REFERENCES users(id),
    agent_id UUID NOT NULL REFERENCES agent_profiles(id),
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    review_text TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_reviews_agent ON reviews(agent_id, created_at DESC);

CREATE TABLE disputes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_id UUID NOT NULL REFERENCES tasks(id),
    opened_by_user_id UUID NOT NULL REFERENCES users(id),
    assigned_admin_id UUID REFERENCES users(id),
    status dispute_status NOT NULL DEFAULT 'OPEN',
    reason VARCHAR(120) NOT NULL,
    description TEXT,
    resolution_note TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at TIMESTAMPTZ
);
CREATE INDEX idx_disputes_status ON disputes(status, created_at DESC);

```

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(160) NOT NULL,  
  body TEXT NOT NULL,  
  data JSONB NOT NULL DEFAULT '{}',  
  read_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ NOT NULL  
-- Full schema is included in the artifact package and appendix.
```

22. API Contract Specification

API Group	Endpoints
Auth APIs	/auth/otp/send, /auth/otp/verify, /auth/refresh, /auth/logout
Customer APIs	/customers/me, /customers/tasks, /customers/notifications
Agent APIs	/agents/register, /agents/me, /agents/location, /agents/availability, /agents/requests
Task APIs	/tasks, /tasks/{id}, /tasks/{id}/accept, /tasks/{id}/contract/confirm, /tasks/{id}/status, /tasks/{id}/complete
Chat APIs	/chat/rooms/{id}/messages, /chat/uploads, Socket.IO events
Call APIs	/calls/session, /calls/external-log, WebRTC signaling events
Review APIs	/reviews, /agents/{id}/reviews
Dispute APIs	/disputes, /disputes/{id}, /admin/disputes
Admin APIs	/admin/agents/pending, /admin/agents/{id}/verify, /admin/users, /admin/tasks, /admin/audit

22.1 REST Standards

- All endpoints are versioned under /v1.
- Use JSON request and response bodies except file uploads, which use multipart/form-data or signed upload policy.
- Return consistent error format with code, message, and details.
- Use pagination for list endpoints with page, limit, sort, and filters.
- Use idempotency keys for task acceptance and message send operations where possible.

22.2 Error Format

```
{
  "code": "TASK_ALREADY_ASSIGNED",
  "message": "This task has already been accepted by another agent.",
  "details": {
    "taskId": "uuid",
    "currentStatus": "ACCEPTED"
  }
}
```

22.3 OpenAPI Excerpt

```
openapi: 3.0.3
info:
  title: EasyDocument API
  version: 1.0.0
  description: API contract for the EasyDocument Nepal document-agent platform.
servers:
  - url: https://api.easydocument.local/v1
security:
  - bearerAuth: []
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    ApiError:
      type: object
      required: [code, message]
      properties:
        code: { type: string }
        message: { type: string }
        details: { type: object }
    SendOtpRequest:
      type: object
      required: [phoneNumber, purpose]
      properties:
        phoneNumber: { type: string, example: "+9779800000000" }
```

```

    purpose: { type: string, enum: [REGISTER, LOGIN, RESET] }
VerifyOtpRequest:
  type: object
  required: [phoneNumber, otp]
  properties:
    phoneNumber: { type: string }
    otp: { type: string }
AuthResponse:
  type: object
  properties:
    accessToken: { type: string }
    refreshToken: { type: string }
    user:
      type: object
CreateTaskRequest:
  type: object
  required: [documentName, organizationName, description]
  properties:
    documentName: { type: string, example: "Passport Renewal" }
    organizationName: { type: string, example: "Department of Passports" }
    organizationAddressText: { type: string }
    organizationLatitude: { type: number, format: double }
    organizationLongitude: { type: number, format: double }
    description: { type: string }
    attachments: { type: array, items: { type: string } }
Task:
  type: object
  properties:
    id: { type: string, format: uuid }
    taskCode: { type: string }
    taskTitle: { type: string }
    status: { type: string }
    assignedAgentId: { type: string, nullable: true }
paths:
  /auth/otp/send:
    post:
      tags: [Auth]
      summary: Send OTP
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/SendOtpRequest' }
      responses:
        '200': { description: OTP sent }
        '429': { description: Rate limited }
  /auth/otp/verify:
    post:
      tags: [Auth]
      summary: Verify OTP and create session
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/VerifyOtpRequest' }
      responses:
        '200':
          description: Authenticated
          content:
            application/json:
              schema: { $ref: '#/components/schemas/AuthResponse' }
  /customers/me:
    get:
      tags: [Customers]
      summary: Get customer profile

```

```

    responses:
      '200': { description: Profile returned }
  patch:
    tags: [Customers]
    summary: Update customer profile
    responses:
      '200': { description: Profile updated }
/agents/register:
  post:
    tags: [Agents]
    summary: Submit agent KYC registration
    responses:
      '201': { description: Agent registration submitted }
/agents/me:
  get:
    tags: [Agents]
    summary: Get agent profile
    responses:
      '200': { description: Agent profile returned }
/tasks:
  post:
    tags: [Tasks]
    summary: Create document task and broadcast to nearby agents
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/CreateTaskRequest' }
    responses:
      '201':
        description: Task created
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Task' }
  get:
    tags: [Tasks]
    summary: List tasks for authenticated user
    responses:
      '200': { description: Task list }
/tasks/{taskId}:
  get:
    tags: [Tasks]
    summary: Get task detail
    parameters:
      - name: taskId
        in: path
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200': { description: Task detail }
/tasks/{taskId}/accept:
  post:
    tags: [Tasks]
    summary: Agent accepts broadcasted task
    responses:
      '200': { description: Task accepted and chat room created }
/tasks/{taskId}/contract/confirm:
  post:
    tags: [Tasks]
    summary: Customer confirms agreed task details and starts task
    responses:
      '200': { description: Task active }
/tasks/{taskId}/status:
  patch:
    tags: [Tasks]
    summary: Agent updates task status and note
    responses:

```

```

    '200': { description: Status updated }
/tasks/{taskId}/complete:
  post:
    tags: [Tasks]
    summary: Customer closes task as completed
    responses:
      '200': { description: Task completed }
/chat/rooms/{roomId}/messages:
  get:
    tags: [Chat]
    summary: Get chat messages
    responses:
      '200': { description: Messages returned }
  post:
    tags: [Chat]
    summary: Send message
    responses:
      '201': { description: Message sent }
/reviews:
  post:
    tags: [Reviews]
    summary: Submit review for completed task
    responses:
      '201': { description: Review created }
/admin/agents/pending:
  get:
    tags: [Admin]
    summary: List pending agent verification requests
    responses:
      '200': { description: Pending agents }
/admin/agents/{agentId}/verify:
  post:
    tags: [Admin]
    summary: Approve or reject agent verification
    responses:
      '200': { description: Verification decision saved }
/admin/disputes:
  get:
    tags: [Admin]
    summary: List disputes
    responses:
      '200': { description: Disputes returned }

```

Full OpenAPI file is included as openapi.yaml in the artifact package.

23. Socket.IO and WebRTC Specification

23.1 Socket Authentication

- Client connects with JWT access token.
- Server validates token and maps socket to user ID and role.
- Joining a room requires task participation or admin dispute permission.
- On disconnect, transient presence is updated in Redis with TTL.

23.2 WebRTC Call Flow

41. Caller taps audio or video call in chat room.
42. Client requests call session from backend.
43. Backend creates call_sessions record with RINGING status.
44. Caller emits call.offer through Socket.IO with SDP offer.
45. Recipient receives incoming call screen and accepts or rejects.
46. Recipient emits call.answer with SDP answer.
47. Both clients exchange ICE candidates through call.ice events.
48. Backend updates call status to ACCEPTED and later ENDED or MISSED.
49. Chat receives system call log message.

23.3 Realtime Failure Handling

- If Socket.IO is unavailable, REST APIs must still allow task status updates and message fetch.
- Unsent chat messages remain in local pending state and retry with client_message_id.
- If WebRTC fails, UI should offer external phone call fallback.
- Call failure is logged with FAILED status for operational visibility.

24. MinIO Object Storage Design

MinIO stores all binary assets including citizenship images, selfies, chat documents, chat images, audio files, and video files. PostgreSQL stores object keys, MIME type, size, uploader, and ownership context.

Bucket	Contents	Access
easydocument-kyc	Citizenship images and selfies	Private; admin and owning agent only
easydocument-chat	Task chat attachments	Private; task participants and dispute admin
easydocument-public	Public assets if needed later	Public read only for non-sensitive assets
easydocument-exports	Admin exports and reports	Private; admin only

24.1 File Upload Rules

- Mobile client requests upload policy or uploads via backend depending on final implementation preference.
- Backend validates MIME type, file extension, size, and ownership context.
- All object keys must be generated by server and include environment prefix.
- Never expose MinIO root credentials to client apps.
- Use signed URLs with short TTL for downloads.

25. DevOps, Environments, and Deployment

25.1 Environments

Environment	Purpose	Data
local	Developer machine using docker-compose	Synthetic seed data
dev	Shared integration environment	Synthetic data only
staging	Production-like QA and release candidate testing	Masked or synthetic data
production	Live customer and agent traffic	Real user data

25.2 Docker Compose Baseline

```
version: "3.9"
services:
  postgres:
    image: postgis/postgis:15-3.4
    environment:
      POSTGRES_DB: easydocument
      POSTGRES_USER: easydoc
      POSTGRES_PASSWORD: easydoc_dev_password
    ports: ["5432:5432"]
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./schema.sql:/docker-entrypoint-initdb.d/001_schema.sql
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    ports: ["9000:9000", "9001:9001"]
    volumes:
      - minio_data:/data
  api:
    build: ../backend
    depends_on: [postgres, redis, minio]
    environment:
      DATABASE_URL: postgresql://easydoc:easydoc_dev_password@postgres:5432/easydocument
      REDIS_URL: redis://redis:6379
      MINIO_ENDPOINT: minio
      MINIO_PORT: 9000
      JWT_SECRET: change-me
    ports: ["3000:3000"]
volumes:
  postgres_data:
  minio_data:
```

25.3 Kubernetes Baseline

- Run at least two API replicas in production.
- Use separate deployments for API and worker processes if background jobs are introduced.
- Use managed or self-hosted PostgreSQL with automated backups.
- Use MinIO with persistent volumes and backup strategy.
- Expose API through Nginx ingress with TLS.
- Use Kubernetes secrets for credentials and API keys.

25.4 CI/CD Pipeline

50. Run lint and format checks for backend, mobile, and admin code.

51. Run backend unit and e2e tests with PostgreSQL and Redis service containers.
52. Run mobile static analysis and widget tests.
53. Build Docker images for backend and admin.
54. Run database migration validation.
55. Deploy to dev automatically after main branch merge.
56. Deploy to staging on release branch.
57. Deploy to production with manual approval and rollback plan.

26. Testing and Quality Strategy

Test Type	Owner	Scope	Minimum Target
Backend unit tests	Backend engineers	Service logic, guards, validators	80 percent service layer coverage
Backend e2e tests	Backend engineers	Auth, task, chat, review, dispute flows	Critical flows covered
Flutter widget tests	Mobile engineers	Forms, task screens, chat states	Critical screens covered
Flutter integration tests	QA/Mobile	End-to-end app flows	Smoke suite for release
Admin tests	Frontend engineers	Agent verification and disputes	Critical admin flows
Load tests	Backend/DevOps	Broadcast and chat traffic	10k target simulation
Security tests	Security/Backend	Auth, RBAC, upload rules	No P0/P1 findings before release

26.1 Critical Test Scenarios

- Customer registers with OTP and creates a task.
- Agent completes KYC and waits for admin approval.
- Admin approves agent and agent receives task broadcast.
- Two agents attempt to accept same task; only one succeeds.
- Chat room is accessible only by assigned customer and agent.
- Customer confirms contract and task becomes active.
- Agent updates status and customer receives notification.
- Customer closes task and submits review.
- Customer opens dispute and admin mediates.
- Unauthorized user cannot access task files or chat messages.

27. Analytics, Monitoring, and Operations

Dashboard	Metrics
Marketplace Health	Requests created, broadcasts sent, acceptance rate, completion rate, cancellation rate
Agent Health	Verified agents, active agents, average rating, response time, dispute count
Customer Health	New customers, repeat customers, task creation conversion, support tickets
System Health	API latency, error rate, database connections, Redis memory, MinIO errors
Dispute Health	Open disputes, first response time, resolution time, reasons

27.1 Alerts

- API 5xx error rate above threshold for 5 minutes.
- PostgreSQL connection saturation.
- Redis unavailable.
- MinIO upload/download failures above threshold.
- OTP provider failure rate spike.
- Task acceptance rate drops sharply in active city.
- Pending agent verification backlog exceeds SLA.

28. Implementation Backlog and Sprint Plan

28.1 Delivery Phases

Phase	Duration	Outcome
Phase 0 - Setup	1 week	Repository, environments, CI skeleton, design system baseline
Phase 1 - Auth and Profiles	2 weeks	OTP, customer profile, agent KYC, admin auth
Phase 2 - Tasks and Matching	3 weeks	Task creation, broadcast, accept, status lifecycle
Phase 3 - Chat and Media	3 weeks	Socket.IO chat, MinIO uploads, file sharing
Phase 4 - Calls and Notifications	2 weeks	WebRTC signaling, external call logs, push notifications
Phase 5 - Admin and Disputes	3 weeks	Agent verification, disputes, user moderation, audit logs
Phase 6 - Reviews and Analytics	2 weeks	Reviews, reputation, analytics dashboards
Phase 7 - Hardening and Launch	3 weeks	Testing, security, performance, store release, production deployment

28.2 Backlog Summary

The full implementation backlog is included as `implementation_backlog.csv` in the artifact package. The table below summarizes the first delivery wave.

Story ID	Epic	Title	Priority	Points
EPIC-01-US-01	Authentication and OTP	Implement Authentication and OTP capability 1	P0	3
EPIC-01-US-02	Authentication and OTP	Implement Authentication and OTP capability 2	P0	5
EPIC-01-US-03	Authentication and OTP	Implement Authentication and OTP capability 3	P0	8
EPIC-01-US-04	Authentication and OTP	Implement Authentication and OTP capability 4	P1	5
EPIC-01-US-05	Authentication and OTP	Implement Authentication and OTP capability 5	P1	3
EPIC-01-US-06	Authentication and OTP	Implement Authentication and OTP capability 6	P1	8
EPIC-01-US-07	Authentication and OTP	Implement Authentication and OTP capability 7	P2	5
EPIC-01-US-08	Authentication and OTP	Implement Authentication and OTP capability 8	P2	3
EPIC-02-US-01	Customer App Core	Implement Customer App Core capability 1	P0	3
EPIC-02-US-02	Customer App Core	Implement Customer App Core capability 2	P0	5
EPIC-02-US-03	Customer App Core	Implement Customer App Core capability 3	P0	8
EPIC-02-US-04	Customer App Core	Implement Customer App Core capability 4	P1	5
EPIC-02-US-05	Customer App Core	Implement Customer App Core capability 5	P1	3
EPIC-02-US-06	Customer App Core	Implement Customer App Core capability 6	P1	8
EPIC-02-US-07	Customer App Core	Implement Customer App Core capability 7	P2	5
EPIC-02-US-08	Customer App Core	Implement Customer App Core capability 8	P2	3
EPIC-03-US-01	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 1	P0	3
EPIC-03-US-02	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 2	P0	5
EPIC-03-US-03	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 3	P0	8
EPIC-03-US-04	Agent Onboarding and Verification	Implement Agent Onboarding and	P1	5

		Verification capability 4		
EPIC-03-US-05	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 5	P1	3
EPIC-03-US-06	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 6	P1	8
EPIC-03-US-07	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 7	P2	5
EPIC-03-US-08	Agent Onboarding and Verification	Implement Agent Onboarding and Verification capability 8	P2	3
EPIC-04-US-01	Task Marketplace and Matching	Implement Task Marketplace and Matching capability 1	P0	3

28.3 Definition of Done

- Feature implemented according to PRD and UX specification.
- Backend APIs documented in OpenAPI where applicable.
- Database migration included and tested.
- Unit tests and critical integration tests pass.
- Mobile states include loading, empty, error, and success handling.
- Audit logs added for sensitive actions.
- Observability logs/metrics added for critical paths.
- Security review completed for auth, RBAC, and file access.

29. AI Coding Agent Instructions

This section is written for Claude Code, Cursor, Windsurf, OpenHands, GitHub Copilot Agent, and similar coding agents. It defines implementation order and guardrails so that generated code stays aligned with product rules.

EasyDocument AI Coding Agent Blueprint

Build Order

1. Create monorepo: backend, mobile, admin, infra, docs.
2. Implement PostgreSQL migrations from schema.sql.
3. Implement NestJS AuthModule with OTP send/verify, JWT access, refresh token rotation.
4. Implement UsersModule and AgentsModule with MinIO KYC upload.
5. Implement TasksModule with broadcast, accept, status history, and direct payment metadata.
6. Implement ChatModule with Socket.IO rooms and message persistence.
7. Implement CallModule with WebRTC signaling and external call log.
8. Implement ReviewsModule and reputation aggregation.
9. Implement React Admin Portal for agent verification and dispute mediation.
10. Add CI/CD, observability, seed data, tests, and release scripts.

Hard Rules

- Do not implement escrow. EasyDocument uses direct payment between customer and agent.
- Do not assume organization registry. The customer manually enters organization name and address.
- Every task status transition must write task_status_history.
- Every admin action must write audit_logs.
- Store files in MinIO; never store file bytes in PostgreSQL.
- Use Google Maps only for map display and geocoding.
- English-only UI for v1.

Repository Layout

```

```
easydocument/
 backend/
 mobile/
 admin/
 infra/
 docs/
 scripts/
```
```

Testing Targets

- Backend unit coverage: minimum 80 percent for service layer.
- Backend e2e: auth, task create, accept, chat, status update, review, dispute.
- Mobile widget tests: onboarding, task creation, task detail, chat.
- Admin integration tests: login, agent approval, dispute update.

29.1 AI Agent Prompt Template

You are implementing EasyDocument, a Nepal-only Flutter + NestJS + PostgreSQL + MinIO platform.

Follow the enterprise implementation package exactly.

Do not add escrow or payment custody.

Do not assume government organization registry.

Use direct customer-to-agent payment metadata only.

Implement module [MODULE_NAME] with DTO validation, RBAC guards, service tests, e2e tests, audit logs where required, and OpenAPI annotations.

Before writing code, inspect existing repository structure and update only the necessary files.

After coding, run lint, tests, and provide a concise implementation summary.

30. Appendices

30.1 Mermaid Sequence Diagram - Task Creation and Acceptance

```
sequenceDiagram
    participant C as Customer App
    participant API as NestJS API
    participant DB as PostgreSQL
    participant M as Matching Service
    participant A as Agent App
    participant CH as Chat Service

    C->>API: POST /tasks
    API->>DB: Insert task CREATED
    API->>M: Find eligible agents
    M->>DB: Insert task_broadcasts
    API->>A: Push task broadcast
    A->>API: POST /tasks/{id}/accept
    API->>DB: Transaction assign agent
    API->>CH: Create chat room
    API->>C: Notify accepted
    C->>CH: Join room
    A->>CH: Join room
```

30.2 Mermaid ERD Overview

```
erDiagram
    users ||--o| agent_profiles : has
    users ||--o{ tasks : creates
    agent_profiles ||--o{ tasks : assigned
    tasks ||--o{ task_broadcasts : broadcasts
    tasks ||--o{ task_status_history : has
    tasks ||--|| chat_rooms : owns
    chat_rooms ||--o{ messages : contains
    chat_rooms ||--o{ call_sessions : has
    tasks ||--o| reviews : receives
    tasks ||--o{ disputes : may_have
    users ||--o{ notifications : receives
    users ||--o{ audit_logs : performs
```

30.3 Environment Variables

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://easydoc:easydoc_dev_password@localhost:5432/easydocument
REDIS_URL=redis://localhost:6379
JWT_SECRET=replace-with-strong-secret
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=30d
MINIO_ENDPOINT=localhost
MINIO_PORT=9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin123
MINIO_BUCKET_ATTACHMENTS=easydocument-attachments
GOOGLE_MAPS_API_KEY=replace-me
SMS_PROVIDER=local-mock
FCM_SERVER_KEY=replace-me
```

30.4 Launch Readiness Checklist

- Production domain and TLS certificate configured.
- PostgreSQL backups verified with restore drill.
- MinIO backup and retention policy configured.
- Firebase push notifications configured for Android and iOS.
- SMS OTP provider configured and rate-limited.

- Google Maps API key configured with app restrictions.
- Admin accounts created with least privilege.
- Agent KYC review SOP approved.
- Dispute mediation SOP approved.
- Privacy policy and terms of service published.
- App store release checklist completed.
- Incident response contacts documented.

30.5 Production Risks and Mitigations

Risk	Impact	Mitigation
Fake agent registration	High trust damage	KYC review, citizenship upload, selfie, phone verification, admin approval
Payment disagreement	Medium to high	Direct payment disclaimer, chat evidence, admin mediation
Agent accepts and disappears	High	Status SLA, support action, cancellation reason, ratings, suspension
Sensitive document leakage	High	Private MinIO, signed URLs, RBAC, audit logs
Broadcast spam	Medium	Task creation rate limits and customer verification
Location inaccuracy	Medium	Google Maps picker confirmation and address fallback
WebRTC unreliable networks	Medium	External call fallback and call failure logs
Operational backlog	Medium	Admin queues and SLA dashboards

31. Detailed API Endpoint Catalog

Method	Path	Authorization	Purpose
POST	/v1/auth/otp/send	Public	Send OTP with rate limits
POST	/v1/auth/otp/verify	Public	Verify OTP and issue tokens
POST	/v1/auth/refresh	Authenticated	Rotate refresh token
POST	/v1/auth/logout	Authenticated	Revoke session
GET	/v1/users/me	Authenticated	Get current user
PATCH	/v1/users/me	Authenticated	Update current user
POST	/v1/agents/register	Authenticated	Submit agent KYC
GET	/v1/agents/me	Agent	Get agent profile
PATCH	/v1/agents/me/availability	Agent	Update availability
PATCH	/v1/agents/me/location	Agent	Update current location
POST	/v1/tasks	Customer	Create task
GET	/v1/tasks	Authenticated	List my tasks
GET	/v1/tasks/{id}	Participant/Admin	Get task
POST	/v1/tasks/{id}/accept	Verified Agent	Accept task
POST	/v1/tasks/{id}/contract/confirm	Customer	Confirm contract
PATCH	/v1/tasks/{id}/status	Assigned Agent	Update status
POST	/v1/tasks/{id}/complete	Customer	Complete task
POST	/v1/tasks/{id}/cancel	Customer/Admin	Cancel task
GET	/v1/chat/rooms/{id}/messages	Participant/Admin	List messages
POST	/v1/chat/rooms/{id}/messages	Participant	Send message
POST	/v1/files/upload-policy	Authenticated	Create upload policy
GET	/v1/files/{id}/signed-url	Authorized	Get signed URL
POST	/v1/calls/sessions	Participant	Create call session
POST	/v1/calls/external-log	Participant	Log external call
POST	/v1/reviews	Customer	Review completed task
GET	/v1/agents/{id}/reviews	Authenticated	List agent reviews
POST	/v1/disputes	Participant	Open dispute
GET	/v1/disputes/{id}	Participant/Admin	Get dispute
POST	/v1/disputes/{id}/messages	Participant/Admin	Add dispute comment
GET	/v1/admin/dashboard	Admin/Ops	Overview metrics
GET	/v1/admin/agents/pending	Admin/Ops	Pending agents
POST	/v1/admin/agents/{id}/verify	Admin/Ops	Approve or reject agent
GET	/v1/admin/tasks	Admin/Ops	Search tasks
GET	/v1/admin/disputes	Admin/Ops	List disputes
PATCH	/v1/admin/disputes/{id}	Admin/Ops	Update dispute
GET	/v1/admin/audit-logs	Admin	Audit logs

32. User Stories and Acceptance Criteria Catalog

Customer Stories

CUS-CAT-001 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 1, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-002 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 2, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-003 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 3, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-004 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 4, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-005 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 5, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-006 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 6, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.

- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-007 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 7, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-008 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 8, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-009 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 9, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-010 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 10, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-011 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 11, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-012 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 12, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-013 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 13, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-014 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 14, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-015 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 15, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-016 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 16, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-017 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 17, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-018 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 18, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-019 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 19, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

CUS-CAT-020 - customer

As a customer, I want to create requests, track progress, communicate, close tasks, and review agents - scenario 20, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

Agent Stories

AGE-CAT-001 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 1, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-002 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 2, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-003 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 3, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-004 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 4, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-005 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 5, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-006 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 6, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-007 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 7, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-008 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 8, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-009 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 9, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-010 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 10, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-011 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 11, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.

- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-012 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 12, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-013 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 13, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-014 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 14, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-015 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 15, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-016 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 16, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-017 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 17, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-018 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 18, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-019 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 19, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

AGE-CAT-020 - agent

As a agent, I want to get verified, accept work, communicate, update progress, and build reputation - scenario 20, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

Admin Stories

ADM-CAT-001 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 1, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-002 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 2, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-003 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 3, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-004 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 4, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-005 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 5, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-006 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 6, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-007 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 7, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-008 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 8, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-009 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 9, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-010 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 10, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-011 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 11, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-012 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 12, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-013 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 13, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-014 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 14, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-015 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 15, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-016 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 16, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.

- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-017 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 17, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-018 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 18, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-019 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 19, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

ADM-CAT-020 - admin

As a admin, I want to verify agents, mediate disputes, protect users, and monitor marketplace health - scenario 20, so that the EasyDocument platform works reliably in real operations.

- Acceptance: Given valid input, the user can complete the action successfully.
- Acceptance: Given invalid input, the system shows a clear validation error.
- Acceptance: Given network failure, the app preserves user context where possible.
- Acceptance: The backend records the action with correct authorization and audit history when required.

33. Complete UI Screen Specification Catalog

This catalog turns product requirements into screen-level build instructions. The Flutter mobile app uses role-aware navigation for customer and agent flows. The React admin portal uses route-protected administrative modules.

CUS-SCR-001 - Splash and Version Gate

Field	Specification
Actor	Customer
Route	/splash
Purpose	Check app version, maintenance mode, auth token, and route target.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-002 - Phone Number Entry

Field	Specification
Actor	Customer
Route	/auth/phone
Purpose	Collect phone number and request OTP.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-003 - OTP Verification

Field	Specification
Actor	Customer
Route	/auth/verify
Purpose	Verify OTP and create authenticated session.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-004 - Customer Profile Setup

Field	Specification
Actor	Customer

Route	/customer/setup
Purpose	Collect full name and address.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-005 - Customer Dashboard

Field	Specification
Actor	Customer
Route	/customer/home
Purpose	Show active tasks, recent history, and create request CTA.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-006 - Create Task Document Step

Field	Specification
Actor	Customer
Route	/tasks/create/document
Purpose	Collect document name, description, and category-like free text.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-007 - Create Task Organization Step

Field	Specification
Actor	Customer
Route	/tasks/create/organization
Purpose	Collect organization name and address.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-008 - Google Map Picker

Field	Specification
Actor	Customer
Route	/tasks/create/map
Purpose	Confirm organization location coordinates.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-009 - Attachment Upload Step

Field	Specification
Actor	Customer
Route	/tasks/create/attachments
Purpose	Upload optional files before broadcast.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-010 - Review and Submit Request

Field	Specification
Actor	Customer
Route	/tasks/create/review
Purpose	Review request and submit.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-011 - Broadcast Waiting

Field	Specification
Actor	Customer

Route	/tasks/:id/waiting
Purpose	Show request broadcast state and allow pre-accept cancel.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-012 - Task Detail

Field	Specification
Actor	Customer
Route	/tasks/:id
Purpose	Show assigned agent, status, timeline, payment metadata, and actions.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-013 - Task Timeline

Field	Specification
Actor	Customer
Route	/tasks/:id/timeline
Purpose	Show historical status updates.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-014 - Chat Room

Field	Specification
Actor	Customer
Route	/chat/:roomId
Purpose	Communicate with assigned agent.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-015 - Contract Confirmation Panel

Field	Specification
Actor	Customer
Route	/tasks/:id/contract
Purpose	Confirm expected date, amount, method, and work start.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-016 - In-App Call Screen

Field	Specification
Actor	Customer
Route	/calls/:callId
Purpose	Handle WebRTC audio/video call.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-017 - External Call Confirmation

Field	Specification
Actor	Customer
Route	/calls/external
Purpose	Open dialer and log external call.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-018 - Close Task

Field	Specification
Actor	Customer

Route	/tasks/:id/complete
Purpose	Customer confirms delivered document and closes task.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-019 - Review Agent

Field	Specification
Actor	Customer
Route	/tasks/:id/review
Purpose	Rate and review agent.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-020 - Open Dispute

Field	Specification
Actor	Customer
Route	/tasks/:id/dispute
Purpose	Submit dispute reason, details, and evidence.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-021 - Dispute Detail

Field	Specification
Actor	Customer
Route	/disputes/:id
Purpose	Track admin mediation updates.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-022 - Task History

Field	Specification
Actor	Customer
Route	/customer/history
Purpose	List completed, cancelled, and disputed tasks.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-023 - Notifications

Field	Specification
Actor	Customer
Route	/notifications
Purpose	List notification history and deep links.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-024 - Customer Profile

Field	Specification
Actor	Customer
Route	/customer/profile
Purpose	Edit customer name/address and view account status.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

CUS-SCR-025 - Support and Safety

Field	Specification
Actor	Customer

Route	/support
Purpose	Safety guidance, contact support, report issue.
Core Components	AppBar, primary CTA, content cards, status chips, bottom navigation where appropriate
Primary APIs	Auth, Tasks, Chat, Calls, Reviews, Disputes, Notifications
States	initial/loading/success/empty/error/offline
Acceptance Criteria	Customer can complete the intended action without leaving the task context.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

33.2 Agent Mobile Screen Specifications

AGT-SCR-001 - Agent Role Selection

Field	Specification
Actor	Agent
Route	/agent/start
Purpose	Allows a user to begin agent registration.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-002 - Agent KYC Identity

Field	Specification
Actor	Agent
Route	/agent/kyc/identity
Purpose	Collect citizenship number and legal identity data.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-003 - Citizenship Upload

Field	Specification
Actor	Agent
Route	/agent/kyc/citizenship
Purpose	Upload citizenship front and back images.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-004 - Selfie Capture

Field	Specification
Actor	Agent
Route	/agent/kyc/selfie
Purpose	Capture or upload agent selfie.
Core Components	Status cards, task cards, map preview, action buttons, upload

	widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-005 - Permanent Address

Field	Specification
Actor	Agent
Route	/agent/kyc/address
Purpose	Collect permanent address text.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-006 - Permanent Location Capture

Field	Specification
Actor	Agent
Route	/agent/kyc/location
Purpose	Capture permanent location using device permission and map confirm.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-007 - KYC Review Submit

Field	Specification
Actor	Agent
Route	/agent/kyc/review
Purpose	Review and submit KYC to admin.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.

- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-008 - Verification Pending

Field	Specification
Actor	Agent
Route	/agent/pending
Purpose	Explain status until admin decision.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-009 - Agent Dashboard

Field	Specification
Actor	Agent
Route	/agent/home
Purpose	Show incoming requests, active tasks, rating, completed tasks.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-010 - Incoming Request Detail

Field	Specification
Actor	Agent
Route	/agent/requests/:id
Purpose	View task summary and accept request.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-011 - Accept Request Confirmation

Field	Specification
Actor	Agent
Route	/agent/requests/:id/accept
Purpose	Confirm acceptance and responsibility.
Core Components	Status cards, task cards, map preview, action buttons, upload

	widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-012 - Agent Task Detail

Field	Specification
Actor	Agent
Route	/agent/tasks/:id
Purpose	Manage assigned task.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-013 - Set Expected Delivery

Field	Specification
Actor	Agent
Route	/agent/tasks/:id/expected-date
Purpose	Set or update expected delivery date.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-014 - Status Update

Field	Specification
Actor	Agent
Route	/agent/tasks/:id/status
Purpose	Update task status and note.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.

- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-015 - Agent Chat Room

Field	Specification
Actor	Agent
Route	/chat/:roomId
Purpose	Communicate with customer.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-016 - Agent Call Screen

Field	Specification
Actor	Agent
Route	/calls/:callId
Purpose	Handle customer call.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-017 - Delivery Evidence Upload

Field	Specification
Actor	Agent
Route	/agent/tasks/:id/delivery
Purpose	Upload delivery or completion evidence.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-018 - Agent Reviews

Field	Specification
Actor	Agent
Route	/agent/reviews
Purpose	View ratings and reviews.
Core Components	Status cards, task cards, map preview, action buttons, upload

	widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-019 - Agent Earnings Metadata

Field	Specification
Actor	Agent
Route	/agent/earnings
Purpose	Show direct payment metadata and completed task amounts.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-020 - Agent Profile

Field	Specification
Actor	Agent
Route	/agent/profile
Purpose	Manage profile, service tags, availability.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-021 - Agent Location Settings

Field	Specification
Actor	Agent
Route	/agent/location-settings
Purpose	Manage location permission and current location update.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.

- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-022 - Agent Dispute Detail

Field	Specification
Actor	Agent
Route	/agent/disputes/:id
Purpose	Respond to dispute and admin requests.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-023 - Agent Notifications

Field	Specification
Actor	Agent
Route	/notifications
Purpose	Receive task and status alerts.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-024 - Agent Safety Guidance

Field	Specification
Actor	Agent
Route	/agent/safety
Purpose	Operational safety instructions.
Core Components	Status cards, task cards, map preview, action buttons, upload widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

AGT-SCR-025 - Agent Support

Field	Specification
Actor	Agent
Route	/support
Purpose	Contact support and report customer issue.
Core Components	Status cards, task cards, map preview, action buttons, upload

	widgets, chat entry
Primary APIs	Agents, Tasks, Chat, Calls, Files, Reviews, Disputes
States	pending/verified/loading/success/error/offline
Acceptance Criteria	Verified agent can perform the intended task operation with correct audit trail.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

33.3 React Admin Screen Specifications

ADM-SCR-001 - Admin Login

Field	Specification
Actor	Admin
Route	/admin/login
Purpose	Authenticate admin user.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-002 - Admin Dashboard

Field	Specification
Actor	Admin
Route	/admin
Purpose	Show platform health and queues.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-003 - Pending Agent Queue

Field	Specification
Actor	Admin
Route	/admin/agents/pending
Purpose	Review new agent applications.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-004 - Agent Verification Detail

Field	Specification
Actor	Admin
Route	/admin/agents/:id/verification
Purpose	Review KYC files and approve/reject.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes

	audit logs.
--	-------------

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-005 - Agent Directory

Field	Specification
Actor	Admin
Route	/admin/agents
Purpose	Search and filter agents.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-006 - Agent Profile Admin View

Field	Specification
Actor	Admin
Route	/admin/agents/:id
Purpose	View agent history and moderation actions.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-007 - Customer Directory

Field	Specification
Actor	Admin
Route	/admin/customers
Purpose	Search and view customers.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-008 - Task Monitor

Field	Specification
Actor	Admin

Route	/admin/tasks
Purpose	Search and inspect tasks.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-009 - Task Admin Detail

Field	Specification
Actor	Admin
Route	/admin/tasks/:id
Purpose	View task, chat metadata, status history.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-010 - Dispute Queue

Field	Specification
Actor	Admin
Route	/admin/disputes
Purpose	List open and assigned disputes.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-011 - Dispute Detail

Field	Specification
Actor	Admin
Route	/admin/disputes/:id
Purpose	Mediate dispute with evidence and notes.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.

- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-012 - Notification Templates

Field	Specification
Actor	Admin
Route	/admin/settings/notifications
Purpose	Manage notification templates.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-013 - Broadcast Settings

Field	Specification
Actor	Admin
Route	/admin/settings/broadcast
Purpose	Manage radius and matching thresholds.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-014 - Audit Log Search

Field	Specification
Actor	Admin
Route	/admin/audit
Purpose	Search audit logs.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-015 - Analytics Overview

Field	Specification
Actor	Admin
Route	/admin/analytics
Purpose	View marketplace analytics.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-016 - System Health

Field	Specification
Actor	Admin
Route	/admin/system
Purpose	View API, database, Redis, MinIO health.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-017 - Reports Export

Field	Specification
Actor	Admin
Route	/admin/reports
Purpose	Generate CSV exports.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-018 - Admin User Management

Field	Specification
Actor	Admin
Route	/admin/users
Purpose	Manage admin accounts and roles.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-019 - Moderation Queue

Field	Specification
Actor	Admin
Route	/admin/moderation

Purpose	Review abuse reports and suspicious activity.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

ADM-SCR-020 - Settings

Field	Specification
Actor	Admin
Route	/admin/settings
Purpose	Configure app policies.
Core Components	Data table, filters, detail drawer, audit notes, confirmation modal
Primary APIs	Admin APIs, Audit APIs, Agent APIs, Task APIs, Dispute APIs
States	loading/success/empty/error/permission-denied
Acceptance Criteria	Admin can complete the operational action and the system writes audit logs.

- The screen must show clear loading, empty, validation, permission, and network-error states.
- All destructive actions must ask for confirmation before submission.
- Analytics event must be emitted for successful primary action and critical failure.
- The UI must remain usable on low-bandwidth networks and small mobile screens.

34. Database Column Catalog

users

Column	Type	Purpose
id	UUID	Primary key
phone_number	VARCHAR(20)	Unique phone login
full_name	VARCHAR(120)	Display and task title
address_text	TEXT	Customer address
role	user_role	CUSTOMER/AGENT/ADMIN/OPS/ FINANCE/SUPPORT
status	user_status	Lifecycle status
profile_photo_url	TEXT	Optional profile image
created_at	TIMESTAMPTZ	Creation time
updated_at	TIMESTAMPTZ	Update time
last_login_at	TIMESTAMPTZ	Last successful login

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.
- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

agent_profiles

Column	Type	Purpose
id	UUID	Primary key
user_id	UUID	Linked user
citizenship_number	VARCHAR(80)	KYC identity number
citizenship_front_url	TEXT	MinIO object URL/key
citizenship_back_url	TEXT	MinIO object URL/key
selfie_url	TEXT	Selfie object URL/key
permanent_address_text	TEXT	Permanent address
permanent_location	GEOGRAPHY	Permanent lat/Ing
current_location	GEOGRAPHY	Current app location
status	agent_status	Verification status
average_rating	NUMERIC	Aggregated rating
completed_task_count	INT	Performance metric
is_available	BOOLEAN	Receives broadcasts

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.
- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

tasks

Column	Type	Purpose
id	UUID	Primary key
task_code	VARCHAR(80)	Unique readable code
customer_id	UUID	Task owner
assigned_agent_id	UUID	Assigned agent
document_name	VARCHAR(120)	Document requested
organization_name	VARCHAR(160)	Manual organization input
organization_address_text	TEXT	Manual address
organization_location	GEOGRAPHY	Geospatial point
task_title	VARCHAR(300)	User-document-organization title
description	TEXT	Instructions
status	task_status	Lifecycle state
direct_payment_method	payment_method	Payment method metadata
agreed_amount_paisa	BIGINT	Amount in paisa
agent_expected_delivery_at	TIMESTAMPTZ	Expected delivery
completed_at	TIMESTAMPTZ	Completion time

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.

- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

messages

Column	Type	Purpose
id	UUID	Primary key
room_id	UUID	Chat room
sender_id	UUID	Message sender
message_type	message_type	TEXT/IMAGE/DOCUMENT/AUDIO/VIDEO/SYSTEM/CALL_LOG
body	TEXT	Text body
attachment_url	TEXT	MinIO object key or URL
attachment_mime	VARCHAR(120)	MIME type
attachment_size_bytes	BIGINT	File size
client_message_id	VARCHAR(120)	Idempotency from mobile
read_at	TIMESTAMPTZ	Read receipt
created_at	TIMESTAMPTZ	Sent time
deleted_at	TIMESTAMPTZ	Soft delete

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.
- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

disputes

Column	Type	Purpose
id	UUID	Primary key
task_id	UUID	Related task
opened_by_user_id	UUID	Customer or agent
assigned_admin_id	UUID	Admin owner
status	dispute_status	Workflow state
reason	VARCHAR(120)	Reason code
description	TEXT	Details
resolution_note	TEXT	Admin decision
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated
closed_at	TIMESTAMPTZ	Closed

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.
- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

audit_logs

Column	Type	Purpose
id	UUID	Primary key
actor_user_id	UUID	Actor
action	VARCHAR(120)	Action name
entity_type	VARCHAR(80)	Entity
entity_id	UUID	Entity ID
ip_address	INET	Source IP
user_agent	TEXT	Client UA
before_data	JSONB	Before snapshot
after_data	JSONB	After snapshot
created_at	TIMESTAMPTZ	Audit time

- All writes to this table must be performed through backend services.
- Foreign key relationships must be enforced by migrations.
- Indexes must be verified using EXPLAIN ANALYZE for high-volume queries.

35. Detailed API Payload Examples

Send OTP

Field	Value
Endpoint	POST /v1/auth/otp/send
Request	{"phoneNumber":"+9779800000000","purpose":"REGISTER"}
Response	{"success":true,"retryAfterSeconds":60}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Verify OTP

Field	Value
Endpoint	POST /v1/auth/otp/verify
Request	{"phoneNumber":"+9779800000000","otp":"123456"}
Response	{"accessToken":"jwt","refreshToken":"jwt","user":{"id":"uuid","role":"CUSTOMER"}}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Create Task

Field	Value
Endpoint	POST /v1/tasks
Request	{"documentName":"Passport Renewal","organizationName":"Department of Passports","organizationAddressText":"Tripureswor, Kathmandu","description":"Need help submitting renewal form."}
Response	{"id":"uuid","taskCode":"RIJAN-PASSPORTRENEWAL-DEPARTMENTOFPASSPORTS","status":"BROADCASTED"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Accept Task

Field	Value
Endpoint	POST /v1/tasks/{id}/accept
Request	{"clientRequestId":"uuid"}
Response	{"taskId":"uuid","status":"ACCEPTED","chatRoomId":"uuid"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Confirm Contract

Field	Value
Endpoint	POST /v1/tasks/{id}/contract/confirm
Request	{"agreedAmountPaisa":250000,"directPaymentMethod":"ESEWA","expectedDeliveryAt":"2026-07-02T12:00:00+05:45"}
Response	{"taskId":"uuid","status":"ACTIVE"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Status Update

Field	Value
Endpoint	PATCH /v1/tasks/{id}/status

Request	{"status":"IN_PROGRESS","note":"Reached organization office and joined queue."}
Response	{"taskId":"uuid","status":"IN_PROGRESS","timelineEntryId":"uuid"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Send Message

Field	Value
Endpoint	POST /v1/chat/rooms/{id}/messages
Request	{"clientMessageId":"local-123","messageType":"TEXT","body":"I have reached the office."}
Response	{"id":"uuid","createdAt":"2026-06-19T10:00:00+05:45"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Create Review

Field	Value
Endpoint	POST /v1/reviews
Request	{"taskId":"uuid","rating":5,"reviewText":"Very helpful and fast."}
Response	{"id":"uuid","agentAverageRating":4.8}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Open Dispute

Field	Value
Endpoint	POST /v1/disputes
Request	{"taskId":"uuid","reason":"AGENT_NOT_RESPONDING","description":"No update for two days."}
Response	{"id":"uuid","status":"OPEN"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

Verify Agent Admin

Field	Value
Endpoint	POST /v1/admin/agents/{id}/verify
Request	{"decision":"APPROVED","note":"Documents match."}
Response	{"agentId":"uuid","status":"VERIFIED"}
Validation	Reject malformed, unauthorized, or state-invalid requests with standard error format.
Audit	Write audit log for security-sensitive or admin actions.

36. QA Test Matrix

Test ID	Area	Scenario	Priority	Pass Criteria
TC-01-01	Auth	Validate auth scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-01-02	Auth	Validate auth scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-01-03	Auth	Validate auth scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-01-04	Auth	Validate auth scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-01-05	Auth	Validate auth scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-02-01	Customer Task Creation	Validate customer task creation scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-02-02	Customer Task Creation	Validate customer task creation scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-02-03	Customer Task Creation	Validate customer task creation scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-02-04	Customer Task Creation	Validate customer task creation scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-02-05	Customer Task Creation	Validate customer task creation scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-03-01	Agent KYC	Validate agent kyc scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-03-02	Agent KYC	Validate agent kyc scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-03-03	Agent KYC	Validate agent kyc scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-03-04	Agent KYC	Validate agent kyc scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-03-05	Agent KYC	Validate agent kyc	P2	Pass when expected

		scenario 5		result matches requirement and no unauthorized side effects occur.
TC-04-01	Agent Verification	Validate agent verification scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-04-02	Agent Verification	Validate agent verification scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-04-03	Agent Verification	Validate agent verification scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-04-04	Agent Verification	Validate agent verification scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-04-05	Agent Verification	Validate agent verification scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-05-01	Matching	Validate matching scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-05-02	Matching	Validate matching scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-05-03	Matching	Validate matching scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-05-04	Matching	Validate matching scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-05-05	Matching	Validate matching scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-06-01	Task Acceptance	Validate task acceptance scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-06-02	Task Acceptance	Validate task acceptance scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-06-03	Task Acceptance	Validate task acceptance scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-06-04	Task Acceptance	Validate task acceptance scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.

TC-06-05	Task Acceptance	Validate task acceptance scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-07-01	Chat	Validate chat scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-07-02	Chat	Validate chat scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-07-03	Chat	Validate chat scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-07-04	Chat	Validate chat scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-07-05	Chat	Validate chat scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-08-01	Media Upload	Validate media upload scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-08-02	Media Upload	Validate media upload scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-08-03	Media Upload	Validate media upload scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-08-04	Media Upload	Validate media upload scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-08-05	Media Upload	Validate media upload scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-09-01	Calls	Validate calls scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-09-02	Calls	Validate calls scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-09-03	Calls	Validate calls scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-09-04	Calls	Validate calls scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.

TC-09-05	Calls	Validate calls scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-10-01	Status Updates	Validate status updates scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-10-02	Status Updates	Validate status updates scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-10-03	Status Updates	Validate status updates scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-10-04	Status Updates	Validate status updates scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-10-05	Status Updates	Validate status updates scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-11-01	Direct Payment Metadata	Validate direct payment metadata scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-11-02	Direct Payment Metadata	Validate direct payment metadata scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-11-03	Direct Payment Metadata	Validate direct payment metadata scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-11-04	Direct Payment Metadata	Validate direct payment metadata scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-11-05	Direct Payment Metadata	Validate direct payment metadata scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-12-01	Reviews	Validate reviews scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-12-02	Reviews	Validate reviews scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-12-03	Reviews	Validate reviews scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-12-04	Reviews	Validate reviews scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.

TC-12-05	Reviews	Validate reviews scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-13-01	Disputes	Validate disputes scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-13-02	Disputes	Validate disputes scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-13-03	Disputes	Validate disputes scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-13-04	Disputes	Validate disputes scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-13-05	Disputes	Validate disputes scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-14-01	Admin Portal	Validate admin portal scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-14-02	Admin Portal	Validate admin portal scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-14-03	Admin Portal	Validate admin portal scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-14-04	Admin Portal	Validate admin portal scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-14-05	Admin Portal	Validate admin portal scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-15-01	Security	Validate security scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-15-02	Security	Validate security scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-15-03	Security	Validate security scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-15-04	Security	Validate security scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.

TC-15-05	Security	Validate security scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-16-01	Notifications	Validate notifications scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-16-02	Notifications	Validate notifications scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-16-03	Notifications	Validate notifications scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-16-04	Notifications	Validate notifications scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-16-05	Notifications	Validate notifications scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-17-01	Offline Handling	Validate offline handling scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-17-02	Offline Handling	Validate offline handling scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-17-03	Offline Handling	Validate offline handling scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-17-04	Offline Handling	Validate offline handling scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-17-05	Offline Handling	Validate offline handling scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-18-01	Performance	Validate performance scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-18-02	Performance	Validate performance scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-18-03	Performance	Validate performance scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-18-04	Performance	Validate performance scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.

TC-18-05	Performance	Validate performance scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-19-01	Audit Logs	Validate audit logs scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-19-02	Audit Logs	Validate audit logs scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-19-03	Audit Logs	Validate audit logs scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-19-04	Audit Logs	Validate audit logs scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-19-05	Audit Logs	Validate audit logs scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-20-01	Release Smoke	Validate release smoke scenario 1	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-20-02	Release Smoke	Validate release smoke scenario 2	P0	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-20-03	Release Smoke	Validate release smoke scenario 3	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-20-04	Release Smoke	Validate release smoke scenario 4	P1	Pass when expected result matches requirement and no unauthorized side effects occur.
TC-20-05	Release Smoke	Validate release smoke scenario 5	P2	Pass when expected result matches requirement and no unauthorized side effects occur.

36.1 Release Regression Pack

- Run full OTP login regression using mock and real SMS provider staging configuration.
- Run task lifecycle from CREATED to COMPLETED.
- Run dispute flow from OPEN to CLOSED.
- Run agent KYC approval and rejection.
- Run file upload and signed URL download authorization tests.
- Run WebRTC call smoke test and external call fallback.
- Run admin RBAC negative tests.

37. Operational SOPs

Agent Verification SOP

- 58. Check phone verification status.
- 59. Review citizenship front and back images for readability.
- 60. Compare selfie and citizenship photo where possible.
- 61. Confirm permanent address and location are plausible.
- 62. Approve, reject with reason, or escalate suspicious submission.
- 63. Every decision must include admin note and audit log.

Dispute Mediation SOP

- 64. Review task status history.
- 65. Review chat and attachments relevant to dispute only.
- 66. Contact both parties through dispute thread if evidence is missing.
- 67. Decide whether task should continue, complete, cancel, or remain suspended.
- 68. Record resolution note and any user/agent moderation action.

Suspicious Agent SOP

- 69. Identify repeated cancellations, low rating, fraud reports, or inconsistent KYC.
- 70. Temporarily suspend if risk is high.
- 71. Notify operations lead.
- 72. Document reason in audit log.
- 73. Restore only after admin review.

Sensitive File Incident SOP

- 74. Disable affected signed URLs if possible.
- 75. Identify tasks and users affected.
- 76. Preserve audit logs.
- 77. Notify leadership and affected users based on policy.
- 78. Patch access-control bug before re-enabling flow.

Production Incident SOP

- 79. Acknowledge alert.
- 80. Classify severity.
- 81. Assign incident commander.
- 82. Mitigate user impact.
- 83. Communicate status internally.
- 84. Create postmortem with action items.

38. Deployment and Release Runbooks

38.1 Backend Release Runbook

85. Confirm all tests pass in CI.
86. Review database migrations for backward compatibility.
87. Deploy to staging and run smoke tests.
88. Verify health endpoints, logs, and metrics.
89. Create production backup before migrations.
90. Deploy production using rolling deployment.
91. Monitor p95 latency, error rate, and database metrics for 60 minutes.
92. Rollback if critical errors exceed threshold.

38.2 Mobile Release Runbook

93. Freeze release branch.
94. Run Flutter analyze, unit tests, widget tests, and integration smoke tests.
95. Verify Android and iOS build signing.
96. Test OTP, task creation, chat, upload, and notification on physical devices.
97. Submit to internal testing track.
98. Run staged rollout.
99. Monitor crash-free sessions and API errors.
100. Expand rollout after stability threshold is met.

38.3 Admin Portal Release Runbook

101. Run TypeScript build and admin integration tests.
102. Verify admin RBAC in staging.
103. Verify agent approval, rejection, dispute update, audit log search.
104. Deploy static/admin container to staging.
105. Promote to production after approval.
106. Monitor frontend errors and API errors.

39. Security Review Checklist

ID	Check	Requirement	Evidence
SEC-001	OTP rate limits	Required before production	Evidence link required in release checklist
SEC-002	JWT expiration	Required before production	Evidence link required in release checklist
SEC-003	Refresh token rotation	Required before production	Evidence link required in release checklist
SEC-004	RBAC on every route	Required before production	Evidence link required in release checklist
SEC-005	Admin action audit logs	Required before production	Evidence link required in release checklist
SEC-006	File MIME validation	Required before production	Evidence link required in release checklist
SEC-007	MinIO private buckets	Required before production	Evidence link required in release checklist
SEC-008	Signed URL TTL	Required before production	Evidence link required in release checklist
SEC-009	Chat room authorization	Required before production	Evidence link required in release checklist
SEC-010	Task participant authorization	Required before production	Evidence link required in release checklist
SEC-011	Agent verified-only task acceptance	Required before production	Evidence link required in release checklist
SEC-012	SQL injection prevention	Required before production	Evidence link required in release checklist
SEC-013	XSS prevention in admin	Required before production	Evidence link required in release checklist
SEC-014	CORS locked down	Required before production	Evidence link required in release checklist
SEC-015	Secrets not committed	Required before production	Evidence link required in release checklist
SEC-016	Logs do not expose OTP or tokens	Required before production	Evidence link required in release checklist
SEC-017	Database backups encrypted	Required before production	Evidence link required in release checklist
SEC-018	KYC access restricted	Required before production	Evidence link required in release checklist
SEC-019	Dispute transcript access restricted	Required before production	Evidence link required in release checklist
SEC-020	Location data minimized	Required before production	Evidence link required in release checklist